



# TUTORIAL: Applicazione E-Commerce Completa in Flutter

Questo progetto realizza un'applicazione di catalogo e carrello spesa interfacciata con le API pubbliche di *Platzi Fake Store*. Utilizza **Provider** per la gestione dello stato globale e **http** per le chiamate di rete.

## 1. Configurazione dell'IDE (VS Code vs Android Studio)



## GUIDA COMPLETA E CODICE SORGENTE: FLUTTER SHOP APP



### Fase 1: Inizializzazione e Configurazione dell'Ambiente

#### se usi VsCode:

1. **Creazione del progetto** Flutter (creerà automaticamente la struttura in Dart):

```
flutter create online_shop_app_flutter
```

2. **Spostarsi nella cartella** del progetto:

```
cd online_shop_app_flutter
```

3. **Installazione dei pacchetti necessari** (senza alcuna forzatura, tramite il CLI ufficiale di Flutter). Ci servono **http** per le chiamate API e **provider** per lo stato globale del carrello:

```
flutter pub add http provider
```

#### ♦ Se usi Android Studio

## 1. Configurazione Iniziale dell'IDE

Prima di creare il progetto, devi assicurarti che Android Studio sia pronto per Flutter (operazione da fare una volta sola):

1. Apri Android Studio.
2. Nella schermata di benvenuto, clicca su **Plugins** nel menu a sinistra.
3. Cerca **Flutter** nella barra di ricerca e clicca su **Install** (installerà automaticamente anche il plugin di Dart).
4. Riavvia l'IDE.

## 2. Creazione del Progetto (Via Interfaccia Grafica)

A differenza di VS Code dove si usa spesso il terminale, in Android Studio puoi fare tutto via interfaccia:

1. Clicca su **New Flutter Project**.
2. Nel menu a sinistra seleziona **Flutter** e assicurati che il percorso della *Flutter SDK path* sia corretto (es. dove hai spaccettato Flutter sul PC). Clicca su *Next*.
3. Imposta i dettagli:
  - **Project name:** `online_shop_app_flutter`
  - **Project type:** `Application`
  - **Languages:** Seleziona *Kotlin* per Android e *Swift* per iOS (sono i default standard).
4. Clicca su **Create**. L'IDE genererà l'intero progetto e aprirà la schermata di lavoro.

## 2. Gestione delle Dipendenze (`pubspec.yaml`)

Prima di scrivere il codice, apri il file `pubspec.yaml` nella root del progetto e assicurati che sotto la voce `dependencies`: siano presenti i due pacchetti fondamentali.

YAML

dependencies:

flutter:

  sdk: flutter

  provider: ^6.1.2

  http: ^1.2.1

Dopo aver salvato il file, digita nel terminale `flutter pub get` per scaricare i pacchetti.

## 3. Struttura dei File e Codice Sorgente Blindato

Crea i file seguendo esattamente questa struttura all'interno della cartella `lib/`:

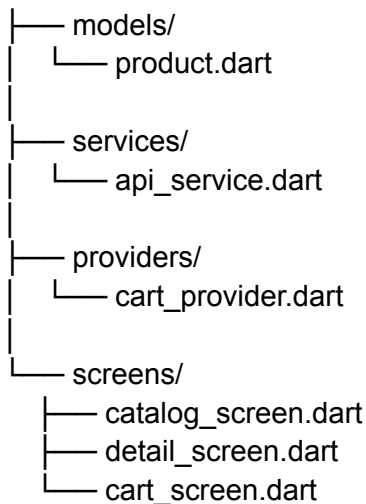
Plaintext

lib/

|

| — main.dart

|



## 1. Il Modello Dati (**lib/models/product.dart**)

*Questo file definisce le strutture dati (**Category**, **Product**, **CartItem**) e mappa i dati JSON in arrivo dall'API tramite i costruttori **factory** di Dart.*

Dart

// Modello per la Categoria del prodotto

```
class Category {
```

```
  final int id;
```

```
  final String name;
```

```
  final String image;
```

```
  Category({required this.id, required this.name, required this.image});
```

// Converti il JSON dell'API in un oggetto Category nativo

```
factory Category.fromJson(Map<String, dynamic> json) {
```

```
  return Category(
```

```
    id: json['id'] ?? 0,
```

```
    name: json['name'] ?? "",
```

```
    image: json['image'] ?? "",
```

```
  );
```

```
}
```

```
}
```

// Modello principale per il Prodotto

```
class Product {
```

```
  final int id;
```

```
  final String title;
```

```
  final double price;
```

```
  final String description;
```

```
  final List<String> images;
```

```
  final Category category;
```

```

Product({
  required this.id,
  required this.title,
  required this.price,
  required this.description,
  required this.images,
  required this.category,
});

// Converte il JSON dell'API in un oggetto Product nativo con controlli di sicurezza
factory Product.fromJson(Map<String, dynamic> json) {
  var jsonImages = json['images'] as List? ?? [];
  List<String> parsedImages = jsonImages.map((img) => img.toString()).toList();

  return Product(
    id: json['id'] ?? 0,
    title: json['title'] ?? "",
    price: (json['price'] as num? ?? 0).toDouble(), // Gestisce sia int che double dal JSON
    description: json['description'] ?? "",
    images: parsedImages,
    category: Category.fromJson(json['category'] ?? {}),
  );
}
}

// Modello per l'elemento all'interno del Carrello (Prodotto + Quantità)
class CartItem {
  final Product product;
  int quantity;

  CartItem({required this.product, this.quantity = 1});
}

```

## 2. Il Servizio API (**lib/services/api\_service.dart**)

*Gestisce la comunicazione HTTP con il server esterno. Include un filtro logico cruciale per scartare i dati corrotti inviati dagli utenti della community sulle API pubbliche di Platzi.*

```

Dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/product.dart';

class ApiService {
  static const String baseUrl = 'https://api.escuelajs.co/api/v1';
}

```

```

// Recupera la lista di tutti i prodotti (limitata a 30 elementi)
static Future<List<Product>> getProducts() async {
  final response = await http.get(Uri.parse('$baseUrl/products?offset=0&limit=30'));

  if (response.statusCode == 200) {
    List<dynamic> body = jsonDecode(response.body);

    // FILTRO ANTI-CRASH: Accetta solo prodotti validi con immagini reali
    List<Product> products = body.map((item) => Product.fromJson(item)).where((product) {
      return product.id != 0 &&
        product.title.isNotEmpty &&
        product.price > 0 &&
        product.category.name.isNotEmpty &&
        product.images.isNotEmpty &&
        product.images[0].startsWith('http');
    }).toList();

    return products;
  } else {
    throw Exception('Impossibile caricare il catalogo prodotti');
  }
}

// Recupera i dettagli di un singolo prodotto specifico tramite ID
static Future<Product> getProductDetails(int id) async {
  final response = await http.get(Uri.parse('$baseUrl/products/$id'));

  if (response.statusCode == 200) {
    return Product.fromJson(jsonDecode(response.body));
  } else {
    throw Exception('Dettagli del prodotto non trovati');
  }
}
}

```

### 3. Lo Stato Globale (**lib/providers/cart\_provider.dart**)

Estende *ChangeNotifier*. Mantiene in memoria gli elementi del carrello e notifica automaticamente i widget della UI tramite *notifyListeners()* ad ogni modifica.

Dart

```

import 'package:flutter/material.dart';
import '../models/product.dart';

class CartProvider with ChangeNotifier {
  // Lista privata degli elementi nel carrello

```

```

final List<CartItem> _cartItems = [];

// Getter pubblico per accedere alla lista in modo sicuro
List<CartItem> get cartItems => _cartItems;

// Aggiunge un prodotto al carrello o ne incrementa la quantità
void addToCart(Product product) {
  int index = _cartItems.indexWhere((item) => item.product.id == product.id);

  if (index != -1) {
    _cartItems[index].quantity += 1;
  } else {
    _cartItems.add(CartItem(product: product, quantity: 1));
  }
  notifyListeners(); // Aggiorna istantaneamente l'interfaccia grafica
}

// Riduce la quantità di un prodotto o lo rimuove se scende a zero
void removeFromCart(int productId) {
  int index = _cartItems.indexWhere((item) => item.product.id == productId);

  if (index != -1) {
    if (_cartItems[index].quantity == 1) {
      _cartItems.removeAt(index);
    } else {
      _cartItems[index].quantity -= 1;
    }
    notifyListeners(); // Aggiorna istantaneamente l'interfaccia grafica
  }
}

// Svuota completamente il carrello
void clearCart() {
  _cartItems.clear();
  notifyListeners();
}

// Calcola il numero totale di pezzi nel carrello
int get totalItemsCount {
  return _cartItems.fold(0, (sum, item) => sum + item.quantity);
}

// Calcola il prezzo totale del carrello
double get totalPrice {
  return _cartItems.fold(0.0, (sum, item) => sum + (item.product.price * item.quantity));
}

```

## 4. Schermata Catalogo (**lib/screens/catalog\_screen.dart**)

*La homepage dell'app. Mostra la lista asincrona dei prodotti tramite un **FutureBuilder** e include una barra fissa in basso con il riepilogo del carrello.*

Dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:online_shop_app_flutter/services/api_service.dart';
import 'package:online_shop_app_flutter/providers/cart_provider.dart';
import 'package:online_shop_app_flutter/models/product.dart';
import 'package:online_shop_app_flutter/screens/detail_screen.dart';
import 'package:online_shop_app_flutter/screens/cart_screen.dart';

// Funzione helper per ripulire le URL delle immagini sporcate dalle stringhe JSON dell'API
String cleanImageUrl(String url) {
  String cleaned = url.replaceAll(RegExp(r'[\[\]]'), '');
  if (!cleaned.startsWith('http')) return 'https://via.placeholder.com/150';
  return cleaned;
}
```

```
class CatalogScreen extends StatelessWidget {
  const CatalogScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // Ascolta lo stato del carrello globale
    final cart = Provider.of<CartProvider>(context);

    return Scaffold(
      appBar: AppBar(title: const Text('Catalogo Prodotti')),
      body: SafeArea(
        child: Column(
          children: [
            // Area principale: Caricamento asincrono del catalogo
            Expanded(
              child: FutureBuilder<List<Product>>(
                future: ApiService.getProducts(),
                builder: (context, snapshot) {
                  if (snapshot.connectionState == ConnectionState.waiting) {
                    return const Center(child: CircularProgressIndicator(color:
Color(0xFF4F46E5)));
                  } else if (snapshot.hasError) {
                    return Center(child: Text('⚠ Errore: ${snapshot.error}', style: const
TextStyle(color: Colors.red)));
                  } else if (!snapshot.hasData || snapshot.data!.isEmpty) {
                    return const Center(child: Text('Nessun prodotto disponibile'));
                  }
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    }

    final products = snapshot.data!;
    return ListView.builder(
      itemCount: products.length, // Usato .length corretto in Dart
      itemBuilder: (context, index) {
        final product = products[index];
        return Card(
          margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
          elevation: 2,
          child: ListTile(
            leading: ClipRRect(
              borderRadius: BorderRadius.circular(8),
              child: Image.network(
                cleanImageUrl(product.images[0]),
                width: 60,
                height: 60,
                fit: BoxFit.cover,
              ),
            ),
            title: Text(product.title, style: const TextStyle(fontWeight: FontWeight.bold)),
            subtitle: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(product.category.name.toUpperCase(), style: const
TextStyle(fontSize: 11, color: Colors.grey)),
                const SizedBox(height: 4),
                Text('€ ${product.price.toStringAsFixed(2)}', style: const TextStyle(color:
Color(0xFF4F46E5), fontWeight: FontWeight.bold)),
              ],
            ),
            onTap: () {
              // Navigazione verso la schermata di dettaglio del prodotto
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => DetailScreen(productId:
product.id)),
              );
            },
          ),
        );
      },
    );
  },
);
// Barra inferiore: Riepilogo rapido del carrello

```



```

Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    border: Border(top: BorderSide(color: Colors.grey.shade300)),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween, // Allineamento strutturale
    children: [
      Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text('${cart.totalItemsCount} prodotti nel carrello', style: const
TextStyle(fontWeight: FontWeight.bold)),
          Text('Totale stimato: € ${cart.totalPrice.toStringAsFixed(2)}', style: const
TextStyle(color: Colors.grey, fontSize: 12)),
        ],
      ),
      ElevatedButton(
        style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFF4F46E5)),
        onPressed: () {
          Navigator.push(context, MaterialPageRoute(builder: (context) => const
CartScreen()));
        },
        child: const Text('Vai al carrello', style: const TextStyle(color: Colors.white)),
      ),
    ],
  ),
),
);
}

```

## 5. Schermata Dettaglio (**lib/screens/detail\_screen.dart**)

*Mostra la scheda completa del singolo articolo con immagine grande, descrizione espansa e il pulsante per aggiungere l'elemento allo stato del carrello.*

Dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:online_shop_app_flutter/services/api_service.dart';

```

```

import 'package:online_shop_app_flutter/providers/cart_provider.dart';
import 'package:online_shop_app_flutter/models/product.dart';

String cleanImageUrl(String url) {
  String cleaned = url.replaceAll(RegExp(r'[\\]'), '');
  if (!cleaned.startsWith('http')) return 'https://via.placeholder.com/300';
  return cleaned;
}

class DetailScreen extends StatelessWidget {
  final int productId;

  // Richiede l'ID del prodotto passato durante la navigazione
  const DetailScreen({Key? key, required this.productId}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // Ottiene il provider senza mettersi in ascolto continuo (listen: false) per ottimizzare le
    performance
    final cartProvider = Provider.of<CartProvider>(context, listen: false);

    return Scaffold(
      appBar: AppBar(title: const Text('Dettaglio Prodotto')),
      body: FutureBuilder<Product>(
        future: ApiService.getProductDetails(productId),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator(color: Color(0xFF4F46E5)));
          } else if (snapshot.hasError || !snapshot.hasData) {
            return const Center(child: Text('Impossibile caricare i dettagli del prodotto.'));
          }
        }
      ),

      final product = snapshot.data!;

      return SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Image.network(cleanImageUrl(product.images[0]), width: double.infinity, height:
300, fit: BoxFit.cover),
            Padding(
              padding: const EdgeInsets.all(20),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(product.category.name.toUpperCase(), style: const TextStyle(color:
Colors.grey, fontWeight: FontWeight.bold, letterSpacing: 1)),
                  const SizedBox(height: 8),

```

```

        Text(product.title, style: const TextStyle(fontSize: 22, fontWeight:
FontWeight.bold)),
        const SizedBox(height: 8),
        Text('€ ${product.price.toStringAsFixed(2)}', style: const TextStyle(fontSize: 20,
color: Color(0xFF4F46E5), fontWeight: FontWeight.bold)),
        const SizedBox(height: 16),
        Text(product.description, style: const TextStyle(fontSize: 15, color:
Colors.black54, height: 1.4)),
        const SizedBox(height: 30),
        // Bottone per aggiungere l'elemento al carrello
        SizedBox(
          width: double.infinity,
          height: 50,
          child: ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: const
Color(0xFF4F46E5), shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12))),
            onPressed: () {
              cartProvider.addToCart(product);
              // Feedback visivo immediato tramite SnackBar
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('${product.title} aggiunto al carrello!'), duration:
const Duration(seconds: 1)),
              );
            },
            child: const Text('🛒 Aggiungi al carrello', style: const TextStyle(fontSize: 16,
color: Colors.white, fontWeight: FontWeight.bold)),
          ),
        ),
        const SizedBox(height: 12),
        SizedBox(
          width: double.infinity,
          height: 50,
          child: OutlinedButton(
            style: OutlinedButton.styleFrom(shape:
RoundedRectangleBorder(borderRadius: BorderRadius.circular(12))),
            onPressed: () => Navigator.pop(context),
            child: const Text('Indietro', style: const TextStyle(fontSize: 16, color:
Colors.black54)),
          ),
        ),
      ],
    ),
  ),
);
},

```

```

    ),
  );
}
}

```

## 6. Schermata Carrello (**lib/screens/cart\_screen.dart**)

*Mostra l'elenco degli articoli selezionati. Permette di aumentare/ridurre le quantità, svuotare la lista e simulare l'invio dell'ordine finale azzerando lo stato globale.*

Dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:online_shop_app_flutter/providers/cart_provider.dart';

class CartScreen extends StatelessWidget {
  const CartScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final cart = Provider.of<CartProvider>(context);

    return Scaffold(
      appBar: AppBar(title: const Text('Il mio Carrello')),
      body: SafeArea(
        child: Column(
          children: [
            // Elenco dei prodotti nel carrello o messaggio di carrello vuoto
            Expanded(
              child: cart.cartItems.isEmpty
                ? const Center(child: Text('Il carrello è vuoto.', style: TextStyle(fontSize: 16, color:
Colors.grey)))
                : ListView.builder(
                    itemCount: cart.cartItems.length,
                    itemBuilder: (context, index) {
                      final item = cart.cartItems[index];
                      return Card(
                        margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
                        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
                        child: Padding(
                          padding: const EdgeInsets.all(16),
                          child: Row(
                            children: [
                              Expanded(
                                child: Column(
                                  crossAxisAlignment: CrossAxisAlignment.start,
                                  children: [

```

```

        Text(item.product.title, style: const TextStyle(fontWeight:
FontWeight.bold, fontSize: 16)),
        Text('Unitario: € ${item.product.price.toStringAsFixed(2)}', style:
const TextStyle(color: Colors.grey, fontSize: 12)),
        const SizedBox(height: 4),
        Text('Subtotale: € ${item.product.price *
item.quantity.toStringAsFixed(2)}', style: const TextStyle(color: Color(0xFF4F46E5),
fontWeight: FontWeight.bold)),
    ],
  ),
),
// Sezione selettore quantità (+ / -)
Row(
  children: [
    IconButton(
      icon: const Icon(Icons.remove_circle_outline),
      onPressed: () => cart.removeFromCart(item.product.id),
    ),
    Text('${item.quantity}', style: const TextStyle(fontSize: 16, fontWeight:
FontWeight.bold)),
    IconButton(
      icon: const Icon(Icons.add_circle_outline),
      onPressed: () => cart.addToCart(item.product),
    ),
  ],
)
],
),
),
);
},
),
),
// Footer: Riepilogo dei costi e pulsanti di checkout
Container(
  padding: const EdgeInsets.all(20),
  decoration: BoxDecoration(color: Colors.white, border: Border(top:
BorderSide(color: Colors.grey.shade300))),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween, // Allineamento nativo
        corretto
        children: [
          const Text('Numero totale prodotti:'),
          Text('${cart.totalItemsCount}', style: const TextStyle(fontWeight:
FontWeight.bold))

```

```

    ],
  ),
  const SizedBox(height: 8),
  Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween, // Allineamento nativo
    children: [
      const Text('Totale complessivo:', style: TextStyle(fontSize: 16)),
      Text('€ ${cart.totalPrice.toStringAsFixed(2)}', style: const TextStyle(fontSize:
18, fontWeight: FontWeight.bold, color: Colors.black))
    ],
  ),
  const SizedBox(height: 16),
  // Pulsante Conferma Ordine
  SizedBox(
    width: double.infinity,
    height: 48,
    child: ElevatedButton(
      style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFF10B981),
shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10))),
      onPressed: cart.cartItems.isEmpty ? null : () {
        // Mostra una finestra di dialogo di successo (protetta dagli apostrofi di crash
        showDialog(
          context: context,
          builder: (context) => AlertDialog(
            title: const Text('Ordine Confermato!'),
            content: const Text('Simulazione d'acquisto completata con successo.'),
            actions: [
              TextButton(
                onPressed: () {
                  cart.clearCart(); // Resetta lo stato globale
                  Navigator.pop(context); // Chiude l>alert
                  Navigator.pop(context); // Torna al catalogo principale
                },
                child: const Text('OK'),
              ),
            ],
          ),
        );
      },
      child: const Text('Conferma ordine', style: TextStyle(color: Colors.white,
fontWeight: FontWeight.bold)),
    ),
  ),
  const SizedBox(height: 8),
  // Pulsante Svuota Carrello
  SizedBox(

```

```

        width: double.infinity,
        height: 48,
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFFEF4444),
shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10))),
          onPressed: cart.cartItems.isEmpty ? null : () => cart.clearCart(),
          child: const Text('Svuota carrello', style: TextStyle(color: Colors.white,
fontWeight: FontWeight.bold)),
        ),
      ),
      const SizedBox(height: 4),
      TextButton(onPressed: () => Navigator.pop(context), child: const Text('Indietro',
style: TextStyle(color: Colors.black54)))
    ],
  ),
)
],
),
),
);
}
}

```

## 7. Il Punto d'Ingresso (**lib/main.dart**)

*Avvia l'app e inietta **CartProvider** in cima all'albero dei componenti tramite **MultiProvider**, rendendo lo stato del carrello accessibile a tutte le pagine.*

Dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:online_shop_app_flutter/providers/cart_provider.dart';
import 'package:online_shop_app_flutter/screens/catalog_screen.dart';

```

```

void main() {
  runApp(
    // Iniezione dello stato globale all'avvio dell'applicazione
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => CartProvider()),
      ],
      child: const MyApp(),
    ),
  );
}

```

```
class MyApp extends StatelessWidget {  
  const MyApp({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Online Shop',  
      debugShowCheckedModeBanner: false, // Rimuove la riga di debug in alto a destra  
      theme: ThemeData(  
        primaryColor: const Color(0xFF4F46E5), // Colore Indigo coordinato  
        scaffoldBackgroundColor: const Color(0xFFF9FAFB),  
        appBarTheme: const AppBarTheme(  
          backgroundColor: Colors.white,  
          foregroundColor: Colors.black,  
          elevation: 1,  
        ),  
      ),  
      home: const CatalogScreen(), // Imposta la pagina del catalogo come schermata iniziale  
    );  
  }  
}
```